

DPN Implementation Workshop – Meridian Financial

Partner Training – Attendee Booklet

August 2026

9 Modules · make-Pipeline Hands-On Lab

Organized around the real `make` targets in `sample/impl`, not product pillars

What changed from the previous booklet: this curriculum replaces the earlier product-pillar structure (Foundation / Three Pillars / DEM / Observability / Shift Left-Right / AI) with modules that map one-to-one onto the actual pipeline stages in the Meridian Financial sample app's Makefile: `make tags` → `make dbm` → `make instrument` → `make dem` → `make security` → `make tf-apply-dd`, plus a fault-injection diagnosis lab and a dedicated AWS/EKS module. There is no DogStatsD anywhere in this curriculum – every custom metric in the sample app is span-based.

Contents

1	Module 0 – Project & Customer Scoping	3
2	Module 1 – Foundation & Deploy	4
3	Module 2 – Unified Service Tagging (make tags)	5
4	Module 3 – Database Monitoring (make dbm)	6
5	Module 4 – APM, Data Streams & Data Jobs Monitoring (make instrument)	7
6	Module 5 – Real User Monitoring (make dem)	9
7	Module 6 – Security (make security)	10
8	Module 7 – Dashboards, Monitors & the Diagnosis Lab (make tf-apply-dd)	11
9	Module 8 – AWS/EKS Deployment	12

Module 0 – Project & Customer Scoping

Before any **make** target runs, a Datadog deployment needs a scope. Meridian Financial is a fictional mid-market financial services company – payments, account management, nightly reconciliation, async fraud detection – used throughout this curriculum.

Learning Objectives

- Read a customer’s architecture for monitoring blind spots before proposing anything
- Map each architecture component to the Datadog capability that closes its gap
- Translate customer pain points into business impact and priority
- Produce a phased, justified proposal – not a feature checklist

Duration: 45–60 minutes **Prerequisites:** none.

The four-step scoping method

1. **Read the Architecture** – critical path, infrastructure beyond the app layer, monitoring gaps, ownership.
2. **Build the Initial Proposal** – map each component to the Datadog capability that closes its blind spot.
3. **Listen to Customer Requirements** – map real pain points to business impact.
4. **Adjust and Prioritize** – phase the proposal: revenue impact first, compliance/risk second, operational efficiency third.

Architecture component	Blind spot	Datadog capability
gateway-api / account-service / transaction-service	No distributed trace across services	APM, Unified Service Tagging
PostgreSQL (postgres-ledger)	Query performance invisible without DBA access	Database Monitoring
ActiveMQ Artemis queues	Can’t tell producer overload from consumer lag	Data Streams Monitoring
batch-processor (Spring Batch)	A job can report success while doing less work	Data Jobs Monitoring
Finance dashboard (frontend)	No visibility into real user sessions	Real User Monitoring
All 6 services	CVEs, injection attacks, container drift	ASM, CWS, CSPM

Hands-On Exercise

Fill in the architecture-to-capability table with one Meridian-specific justification per row. Pick two customer pain points and assign each a priority phase with one success criterion. *Expected outcome:* a one-page phased proposal grounded in the real architecture, mapping onto Modules 1–8.

Datadog Documentation

- Getting Started with Datadog: https://docs.datadoghq.com/getting_started/
- Getting Started with Tagging: https://docs.datadoghq.com/getting_started/tagging/

Module 1 – Foundation & Deploy

Org setup is general and applies to any customer; deploying Meridian Financial “cold” – with every Datadog signal off – is the starting point every later module instruments from.

Learning Objectives

- Configure a new Datadog org: authentication, users, teams, API/App keys
- Understand when to use single vs. multi-org architecture
- Deploy the app with zero instrumentation active and confirm it runs cleanly
- Install the Datadog Agent without enabling any instrumentation layer yet

Duration: 1–1.5 hours **Prerequisites:** Datadog sandbox org (Admin), a local cluster or AWS account, kubectl/helm/terraform.

Organization setup

Key type	Purpose	Who holds it
API key	Authenticates data sent to Datadog (Agent, Terraform)	The org – secrets manager
App key	Authenticates API calls on a user's behalf	Each individual user

Get the App Key *value*, not the Key ID – the list view shows the Key ID by default. Using the Key ID causes 401 Unauthorized on every terraform apply.

Deploy Meridian Financial cold

```
make build          # build all 6 service images + traffic-generator
make deploy-k8s     # deploy the app into the finance namespace
make deploy-k8s-dd  # install Operator, create secret, deploy the Agent
```

Confirm: `kubectl get pods -n finance` (12 Running), log in to the dashboard at `http://localhost:30080` as `carol.admin / Finance@2025!`. At this point: no traces, no DBM, no RUM – the deliberate starting state.

Hands-On Exercise

Complete the org setup decisions for your sandbox. Deploy the app and Agent cold. Confirm APM > Services shows nothing yet (expected). *Expected outcome:* a running baseline deployment with a documented org configuration.

Datadog Documentation

- Managing Multiple Organizations: https://docs.datadoghq.com/account_management/multi_organization/
- SAML SSO: https://docs.datadoghq.com/account_management/saml/
- Teams: https://docs.datadoghq.com/account_management/teams/
- API & Application Keys: https://docs.datadoghq.com/account_management/api-app-keys/

Module 2 – Unified Service Tagging (make tags)

Learning Objectives

- Design a tag taxonomy grounded in real business services
- Apply UST + log injection to all six services and verify correlation end-to-end
- Recognize each language's log-injection mechanism – Python, Node, Java, Go all differ

Duration: 45–60 minutes **Prerequisites:** Module 1 complete.

Language	Service(s)	Log-injection mechanism
Python	gateway-api, fraud-detection	ddtrace.contrib.logging.patch + patch_logging()
Node.js	transaction-service	logInjection: true in the dd-trace init block
Java	account-service, batch-processor	DD_LOGS_INJECTION=true (Logback/Log4j2 MDC hook)
Go	notification-service	Manual dd.trace_id/dd.span_id injection – no automatic MDC hook

Log-collection annotation is a separate sub-step. `make tags` also uncomments the `ad.datadoghq.com/<service>.logs` Agent annotation on all six manifests – this is what tells the Agent which `source/service` to tag a pod's collected logs with, for correct pipeline parsing and faceting. It's independent of the UST env vars and log-injection blocks above: without it, logs still arrive in Log Explorer, just without correct `source` faceting.

transaction-service stamps UST fields into its own logs too. Its `make tags` patch has a second hunk for `index.js`, uncommenting the pino logger's `base: { service, env, version }` object – the same UST fields, written directly onto every JSON log line by the app itself instead of relying on the Agent to infer them.

Ordering matters for Go. `notification-service`'s log injection reads the `alert.send` span's trace/span ID – a span that only exists once Module 4's `make instrument` has uncommented it. Running `make tags` alone first leaves an undefined `span` variable and the build fails.

```
make tags && make build && make deploy-k8s    # apply UST + log injection, rebuild,
    redeploy
make untag && make build && make deploy-k8s    # reverse
```

Validate: any trace/log carries `env:staging service:<name> version:latest`. In Log Explorer, a **View Trace** button appears on any log once `dd.trace_id` is present.

Hands-On Exercise

Apply UST + log injection. Confirm env vars are set on each pod. Prove the log→trace jump works for at least one service, then reverse it.

Datadog Documentation

- Unified Service Tagging: https://docs.datadoghq.com/getting_started/tagging/unified_service_tagging/
- Log-Trace Correlation: https://docs.datadoghq.com/tracing/other_telemetry/connect_logs_and_traces/
- Tag Policies: <https://docs.datadoghq.com/monitors/settings/>

Module 3 – Database Monitoring (make dbm)

Learning Objectives

- Understand what DBM collects and why it needs its own database role
- Apply `make dbm` and confirm query metrics and explain plans appear
- Understand the APM ↔ DBM correlation link

Duration: 30–45 minutes **Prerequisites:** Module 2 complete.

Two steps: (a) Agent-side config uncomments the `postgres.d` check + password wiring; (b) a dedicated read-only `datadog` role is created in PostgreSQL, granted `pg_stat_statements` and the `datadog.explain_statement` function. The monitoring role is **read-only** (`pg_monitor` only, never superuser) – important for a ledger database specifically.

```
make dbm
kubectl apply -k deploy/kubernetes/datadog/agent
kubectl rollout restart daemonset/datadog-agent -n datadog
make undbm      # reverse: drops the role, reverses the Agent-side patch
```

Validate: Databases → Query Metrics shows `postgres-ledger` queries; open a sample → **Explain Plan**.

Hands-On Exercise

Enable DBM. Generate payment traffic. Open a `POST /v1/payments` trace, find the `ledger.velocity_check` span, click **View in DBM**, confirm it lands on that exact query's explain plan. Reverse it.

Datadog Documentation

- Database Monitoring: https://docs.datadoghq.com/database_monitoring/
- DBM – PostgreSQL self-hosted setup: https://docs.datadoghq.com/database_monitoring/setup_postgres/selfhosted/
- DBM + APM correlation: https://docs.datadoghq.com/database_monitoring/connect_dbm_and_apm/

Module 4 – APM, Data Streams & Data Jobs Monitoring (make instrument)

The densest module – four steps under one sentinel: APM custom spans, Single Step Instrumentation gating, Continuous Profiler, and DSM/DJM.

Learning Objectives

- Understand which spans are always-on vs. patch-gated, per service
- Enable Single Step Instrumentation and verify tracer injection actually happened
- Correlate a Continuous Profiler flame graph with a slow trace
- Understand DSM's producer-vs-consumer distinction – including a real STOMP coverage gap
- Understand what Data Jobs Monitoring adds on top of a job's own status field

Duration: 1.5–2 hours **Prerequisites:** Modules 2 and 3.

No DogStatsD anywhere. Every `finance.*` custom metric is span-based (`datadog_spans_metric`), generated directly from the spans below.

The STOMP / DSM gap

`transaction-service` publishes to `fraud.score.queue` over STOMP via the `stompit` library – and `dd-trace` Node.js has **no automatic STOMP instrumentation at all**. Without extra work it never appears as a producer in the DSM pathway map, no matter how much traffic it generates. `make instrument` closes this with a **manual DSM checkpoint** (`setProduceCheckpoint`) in `producer.js`, plus a matching manual consume checkpoint in `fraud-detection/listener.py`. Module 7's diagnosis lab Scenario 3 depends entirely on this checkpoint being in place.

Service	Library	Injection mechanism
gateway-api, fraud-detection	ddtrace (Python)	PYTHONPATH + auto-instrumentation
transaction-service	dd-trace (Node.js)	NODE_OPTIONS=-require dd-trace/init
account-service, batch-processor	dd-java-agent	JAVA_TOOL_OPTIONS=-javaagent:...
notification-service	dd-trace-go	Not single-step injected – in-code <code>tracer.Start()</code>

gateway-api and fraud-detection never get a pip-installed ddtrace. Unlike every other service here, their `requirements.txt` permanently omits it – not just commented out until `make instrument`, but never pinned at all. Single Step Instrumentation plus the Datadog Operator installed in-cluster (`make deploy-k8s-dd`) are the *only* source of `ddtrace` for these two services; without SSI actually injecting it and the pod redeployed, Step 1's uncommented imports and custom spans raise `ModuleNotFoundError`. This also closes a previously-documented bug: a baked-in pip copy could no longer silently shadow the SSI-injected one and make the `python-lib.version` annotation a no-op, because there is no baked-in copy to shadow it with.

transaction-service's dd-trace APM init is patch-gated too. `require('dd-trace').init()` in `index.js`, plus the `dd-trace` npm dependency in `package.json`, are commented out by default and only added by `make instrument` – matching how `notification-service`'s Go tracer already worked.

fraud-detection needs a rebuild, not just a redeploy. Its DSM support is a baked pip dependency, unlike the other services' plain env-var toggle. After `make instrument`, run `make build` (or `build-ecr`) and reload the image before DSM data appears for it.

```
make instrument && make build && make deploy-k8s
make uninstrument && make build      # reverse: DSM/DJM -> profiler -> SSI -> APM spans
```

Hands-On Exercise

Enable all four steps. Verify custom spans, SSI injection, profiler flame graphs. Confirm `transaction-service` appears as a **producer** in the DSM pathway map for `fraud.score.queue` – this is the check Module 7 Scenario 3 depends on. Trigger a reconciliation run and confirm it appears under APM → Data Jobs.

Datadog Documentation

- Custom instrumentation: https://docs.datadoghq.com/tracing/trace_collection/custom_instrumentation/
- Single-step instrumentation: https://docs.datadoghq.com/tracing/trace_collection/automatic_instrumentation/single-step-apm/
- Continuous Profiler: <https://docs.datadoghq.com/profiler/>
- Data Streams Monitoring: https://docs.datadoghq.com/data_streams/
- Data Streams – Node.js manual checkpoints: https://docs.datadoghq.com/data_streams/nodejs/
- Data Jobs Monitoring: https://docs.datadoghq.com/data_jobs/

Module 5 – Real User Monitoring (make dem)

Learning Objectives

- Understand how `make dem` differs mechanically – a direct API call, not a patch
- Enable Browser RUM + Session Replay and confirm a session is captured
- Apply PII masking correctly for a financial application's session replay

Duration: 30–45 minutes **Prerequisites:** App running (Module 1). Independent of Modules 2–4 and 7 – no Terraform dependency.

The RUM `<script>` block in `frontend-stub/index.html` ships commented out inside an HTML comment, like every other Datadog block in this repo – the page has zero Datadog code at rest. `make dem` creates the RUM application via a direct POST `/api/v2/rum/applications` call, uncomments that `<script>` block, and injects the resulting `client_token` with `sed` – previously it only did the credential injection, since the script was already active. Idempotent via `.dem-applied`.

PII masking matters specifically here – this is a financial dashboard. `defaultPrivacyLevel: 'mask-user-input'` ensures form values (account numbers, payment amounts) are never captured in Session Replay. Never disable this on real financial data.

```
make dem
kubectl create configmap frontend-dashboard
  --from-file=index.html=frontend-stub/index.html -n finance --dry-run=client -o
  yaml | kubectl apply -f -
kubectl rollout restart deployment/frontend -n finance
make undem # reverse
```

Validate: RUM → Applications → `finance-frontend` → Sessions → Session Replay available.

Hands-On Exercise

Enable RUM. Click through the dashboard. Confirm Session Replay shows the interaction with form fields masked, not literal values. Reverse it.

Datadog Documentation

- Browser RUM: https://docs.datadoghq.com/real_user_monitoring/browser/
- RUM Session Replay: https://docs.datadoghq.com/real_user_monitoring/session_replay/
- RUM Privacy / PII masking: https://docs.datadoghq.com/real_user_monitoring/session_replay/privacy_options/

Module 6 – Security (make security)

Learning Objectives

- Understand the two-sided nature of **make security** – Agent config + app-side flag
- Distinguish what ASM Threats, ASM SCA, CWS, and CSPM each detect
- Enable the stage and confirm CWS self-tests pass

Duration: 30–45 minutes **Prerequisites:** Module 4 – ASM Threats needs the tracer already injected via Single Step Instrumentation.

Product	Layer	Detects
ASM Threats	APM tracer (app-side)	SQLi, XSS, SSRF, credential stuffing, business-logic attacks
ASM SCA	Agent-side	Known CVEs in dependencies
CWS	Agent eBPF (kernel)	Shell spawned in container, privilege escalation, syscall anomalies
CSPM	Agent + cloud APIs	Privileged pods, exposed secrets, CIS/PCI-DSS findings

```
make security
kubectl apply -k deploy/kubernetes/datadog/agent && kubectl rollout restart
  daemonset/datadog-agent -n datadog
make build && make deploy-k8s    # DD_APPSEC_ENABLED needs a redeploy, not a bare
  restart
make unsecurity    # reverse
```

Validate: DD_APPSEC_ENABLED=true on gateway-api; security-agent status shows CWS self-tests succeeded (rule_open, rule_chmod, rule_chown).

Hands-On Exercise

Enable all four products. Confirm both halves independently. Configure one finance-specific threat rule (e.g. brute force on /v1/payments). Reverse it.

Datadog Documentation

- Application Security (ASM): https://docs.datadoghq.com/security/application_security/
- Cloud Workload Security (CWS): https://docs.datadoghq.com/security/cloud_workload_security/
- Cloud Security Posture Management (CSPM): https://docs.datadoghq.com/security/cloud_security_management/misconfigurations/

Module 7 – Dashboards, Monitors & the Diagnosis Lab (make tf-apply-dd)

Learning Objectives

- Apply the Terraform-managed dashboard, monitors, SLOs, and synthetic tests
- Diagnose three realistic incidents live, using the exact capability that catches each one
- Understand why a monitor's absolute thresholds can fail to fire at demo scale even when the underlying signal is genuinely correct

Duration: 1.5–2 hours **Prerequisites:** Modules 2–4.

```
make tf-apply-dd      # DD_API_KEY / DD_APP_KEY read from .env automatically;
  dashboard,          # 7+ monitors, 3 SLOs, 7 synthetic tests, log pipeline
make tf-destroy-dd    # reverse -- WARNING: deletes the log index and indexed logs
  too
```

Scenario 1 – Payments Are Slow and No One Knows Why

make scenario-1 drops `idx_transactions_account_id`, turning the `ledger.velocity_check` query into a full table scan on every payment. The `transactions` table is pre-seeded with ~300k backdated rows specifically so this scan is slow enough to see – on live traffic volume alone it finishes in under a millisecond. **Capability:** APM traces + log correlation + the APM ↔ DBM link.

Scenario 2 – The Nightly Reconciliation Is Failing Silently

make scenario-2 appends AND `currency <> 'JPY'` to the reconciliation reader's WHERE clause – the job still completes with no error; only the record count drops ~20%. **Capability:** Data Jobs Monitoring.

The `reconciliation_low_record_count` monitor's absolute thresholds were sized for production volume – at workshop traffic scale, the ~20% drop may not cross them within the session. `job.records_processed` in Data Jobs Monitoring is the live, always-reliable signal; treat the monitor as “here's the kind of alert you'd build,” not a guaranteed live-fire.

Scenario 3 – The Fraud Queue Is Backing Up

make scenario-3 triples `transaction-service`'s publish rate to `fraud.score.queue`. The producer floods the queue; `fraud-detection` keeps up at its normal rate – scaling the consumer would do nothing. **Depends on Module 4's DSM manual checkpoint** – without it `transaction-service` never appears as a producer at all. **Capability:** Data Streams Monitoring pathway map + Deployment Tracking.

The `fraud_queue_depth` monitor may **not** fire at workshop scale – `fraud-detection` keeps up even at 3x load, so depth stays near 0. The `fraud_queue_producer_surge` monitor (a change alert on the production *rate*, not depth) is the one that actually catches this scenario live.

Hands-On Exercise

Run all three scenarios in turn. Diagnose each using only the intended capability, before checking this booklet's answer. Note explicitly which monitors fired and which didn't, and why. Reverse all three, then `tf-destroy-dd` if ending the session.

Datadog Documentation

- Synthetic Monitoring: <https://docs.datadoghq.com/synthetics/>
- Synthetic → APM correlation: <https://docs.datadoghq.com/synthetics/apm/>
- Generate metrics from spans: https://docs.datadoghq.com/tracing/trace_pipeline/generate_metrics/

Module 8 – AWS/EKS Deployment

Learning Objectives

- Provision EKS/VPC/ECR/IAM via Terraform and confirm the cluster is reachable
- Understand the two AWS-specific integration points this app needs
- Tear down AWS resources in the correct order

Duration: 1–1.5 hours **Prerequisites:** AWS CLI $\geq$ 2.x, an SSO profile, Terraform $\geq$ 1.5.

RUM validation status: Module 5’s Browser RUM was validated locally in this curriculum’s development. **It has not been validated end-to-end on AWS EKS** – treat it as untested territory on this path, and verify it yourself before relying on it with a customer.

```
aws sso login --profile <profile>
make tf-plan-aws && make tf-apply-aws      # provisions EKS/VPC/ECR/IAM/NLB, ~15-20
min
make tf-configure-kubectl && kubectl get nodes
make build-ecr && make deploy-k8s-eks
```

Two AWS-specific integration points Terraform doesn’t own: Keycloak’s public URL (patch the `app-config` ConfigMap and the frontend HTML to point at the NLB) and Keycloak’s allowed CORS origins (the realm import hardcodes `localhost` – patch via `kcadm.sh` after the NLB hostname is known). Both are detailed in Module 8’s full write-up (`docs/Module-8-AWS-EKS-Deployment.md`).

Both patches are **in-memory only** – Keycloak runs `start-dev` with an in-process H2 database, so any pod restart wipes them back to `localhost-only`. If login fails with `Failed to fetch` again after working before, re-apply the CORS patch.

```
make deploy-k8s-dd
make tf-apply-dd
# Teardown, in this order:
make tf-destroy-dd
make tf-destroy-aws
```

Hands-On Exercise

Provision, deploy, patch Keycloak’s URL and CORS origins, log in via the NLB URL, apply Datadog resources, then tear down completely. *Expected outcome:* a working EKS deployment confirmed end-to-end, followed by a clean teardown with no orphaned AWS resources.

Datadog Documentation

- AWS Integration: https://docs.datadoghq.com/integrations/amazon_web_services/
- Datadog on Kubernetes (Operator/Helm, Cluster Agent): <https://docs.datadoghq.com/containers/kubernetes/>
- Datadog Operator: <https://github.com/DataDog/datadog-operator>
- Terraform provider: <https://registry.terraform.io/providers/DataDog/datadog/latest/docs>

Source of truth for all modules: `sample/impl/INSTRUMENTATION.md`, `USECASES.md`, `README.md`, `TROUBLESHOOTING.md` – this booklet summarizes, the repo governs.